Trabalho Prático 2: Busca Local e Algoritmo Genético

Artur Filardi Victoriano
2023060707

Bernardo Soares Diniz Lara
2023060898

Germano Marques Cipriano Fagundes
2023060839

Resumo

Comparamos três metaheurísticas – *Greedy Descent* com reinício aleatório (GD), *Simulated Annealing* (SA) e Algoritmo Genético (GA) – no problema de sequenciar n manutenções para minimizar o atraso ponderado total, em instâncias com $n \in \{12, 50, 150\}$. Todos os métodos recebem o mesmo orçamento de avaliações da função objetivo e as mesmas sementes aleatórias. Em $n = 12$, GD e SA atingem o ótimo global (1 875) em todas as execuções; com o aumento da escala ($n = 50$ e 150), o SA passa a superar o GD ($p = 5{,}8 \times 10^{-4}$ em $n = 150$, chegando a superar o valor de referência do enunciado em $n = 50$), enquanto ambos superam significativamente o GA, que sofre perda rápida de diversidade e baixa eficácia do cruzamento OX diante do acoplamento temporal dos prazos.

1 Formulação do Problema e Validação (T1)

Uma equipe parte da base (nó 0) e deve executar a manutenção de n ativos, um de cada vez, sem retornar à base ao final. O ativo j fica em (x_j, y_j), exige p_j unidades de tempo de serviço, tem prazo d_j e peso de criticidade w_j. O deslocamento entre i e j leva $t_{ij} = \text{round}\big(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\big)$, com a matriz pré-calculada. Uma solução é uma permutação $\pi = (\pi_1, \ldots, \pi_n)$ dos ativos, com $\pi_0 = 0$, $C_0 = 0$ e

$$C_k = C_{k-1} + t_{\pi_{k-1}\pi_k} + p_{\pi_k},$$
$$h(\pi) = \sum_{k=1}^{n} w_{\pi_k} \max\{0,\ C_k - d_{\pi_k}\}, \qquad (1)$$

o atraso ponderado total, a ser minimizado. O problema opõe dois critérios. Visitar primeiro os ativos próximos reduz o tempo gasto em deslocamento, mas pode deixar para depois um ativo distante, urgente (prazo curto) e crítico (peso alto). Atender os urgentes primeiro, por sua vez, pode gerar rotas em zigue-zague, que atrasam todos os ativos seguintes. Como C_k se acumula, uma decisão no início da rota afeta o atraso de todo o sufixo.

Validação (T1). O script `scripts/tarefa_t1_validacao_h.py` avalia h pela interface usada pelos algoritmos e por uma implementação direta independente, e reproduz os três valores de referência do enunciado: $h(1, 4, 2, 3) = 39$ e $h(1, 4, 3, 2) = 22$ (ótima) no exemplo da Figura 1, e $h = 4\,366$ para a identidade $(1, \ldots, 12)$ na instância pequena. O `round` do Python arredonda empates x,5 para o par. Verificamos que nenhuma distância nas três instâncias cai num empate, de modo que t_{ij} coincide com o arredondamento usual.

2 Algoritmos e Estratégias de Busca

2.1 Vizinhanças e Avaliação Incremental (T2)

Um movimento é definido por duas posições $i \neq j$ em $\{1, \ldots, n\}$. O *Swap* troca π_i e π_j e gera $n(n-1)/2$ vizinhos.

Tabela 1. Custo por avaliação de um vizinho (μs) num ótimo local: avaliação completa, incremental (sufixo) e incremental com poda.

Inst.	Viz.	$\lvert V\rvert$	compl. (μs)	incr. (μs)	+poda (μs)	speedup incr.	speedup +poda
Pequena	Swap	66	1,58	1,32	1,25	1,20×	1,27×
	Insertion	121	1,57	1,34	1,23	1,17×	1,28×
	2-opt	66	1,59	1,38	1,21	1,16×	1,32×
Intermediária	Swap	1 225	5,23	3,88	3,54	1,35×	1,48×
	Insertion	2 401	4,85	3,77	4,11	1,29×	1,18×
	2-opt	1 225	4,87	3,65	3,44	1,33×	1,42×
Grande	Swap	11 175	14,19	10,29	9,93	1,38×	1,43×
	Insertion	22 201	13,03	9,81	9,47	1,33×	1,38×
	2-opt	11 175	13,51	11,86	9,98	1,14×	1,35×

A *Insertion* retira π_i e o reinsere na posição j. Como mover i para $i-1$ equivale a mover $i-1$ para i, ela tem $(n-1)^2$ vizinhos distintos. O *2-opt* inverte o trecho $\pi_i, \ldots, \pi_j$ e gera $n(n-1)/2$ vizinhos. Em $n = 150$, são 11 175, 22 201 e 11 175 vizinhos, respectivamente; um teste automático confere essas contagens e que os vizinhos são distintos.

Avaliação incremental. Um movimento com menor posição alterada $k_{\min} = \min(i, j)$ não altera $C_1, \ldots, C_{k_{\min}-1}$ nem o atraso acumulado $H_k = \sum_{l \le k} w_{\pi_l} \max\{0, C_l - d_{\pi_l}\}$ desse prefixo. Guardamos os vetores C e H da solução corrente, e o vizinho é avaliado percorrendo só o sufixo $k_{\min}, \ldots, n$, sem alocar vetores. Eles são reconstruídos apenas quando o movimento é aceito, sem custo de orçamento, pois o h do vizinho já foi contado. No GD usamos ainda uma *poda*: como H_k é não decrescente, a varredura para assim que H_k atinge o valor a ser batido. O teste automático compara a avaliação incremental com a completa em 9 000 movimentos aleatórios (3 vizinhanças × 3 instâncias) e verifica que a poda nunca descarta uma melhora.

A Tabela 1 mostra o custo por avaliação, medido sobre toda a vizinhança de um ótimo local. O ganho é modesto, de 1,1 a 1,5×. Com i, j uniformes, $\mathbb{E}[k_{\min}] \approx n/3$, então o sufixo ainda cobre cerca de $2/3$ da rota, e há um custo fixo por chamada. A poda acrescenta pouco, pelo próprio formato do problema: no início da rota quase nenhum prazo venceu, e o atraso se concentra nos últimos ativos. H_k só ultrapassa o limiar perto do fim, e a varredura raramente é interrompida cedo.

Comparação das vizinhanças (T2). A Tabela 2 compara as três vizinhanças no GD com reinício. Na instância pequena, todas atingem o ótimo conhecido $h = 1\,875$. Nas maiores, a Insertion é claramente superior: em $n = 150$, a mediana é 80 011, contra 90 544 (2-opt) e 147 824 (Swap), e seus ótimos locais também são os melhores em média. A explicação está na estrutura do problema. Antecipar um ativo urgente ou adiar um pouco urgente é exatamente um movimento de inserção, que preserva a ordem relativa dos demais. O Swap desloca dois ativos ao mesmo tempo, e o 2-opt inverte a ordem de um bloco inteiro. Essa operação é natural no

Tabela 2. T2: GD (First Improvement) com reinício por vizinhança, 10 runs com o orçamento B do T3. $\tilde{h}^\star$: mediana do melhor h; $\bar{h}_{\text{ót.loc.}}$: h médio dos ótimos locais; ót. loc. e passos: por run.

Inst.	Viz.	$\tilde{h}^\star$	$\bar{h}_{\text{ót.loc.}}$	ót. loc.	passos	$\bar{t}$ (s)
Pequena	Swap	1 875	2 569	257,3	4 036	0,08
	Insertion	1 875	2 199	147,7	2 605	0,08
	2-opt	1 875	2 151	244,7	4 227	0,09
Intermediária	Swap	22 882	30 709	24,3	3 142	0,50
	Insertion	17 048	20 361	12,0	1 916	0,53
	2-opt	18 124	21 890	23,6	3 294	0,58
Grande	Swap	147 824	163 234	3,3	2 212	3,03
	Insertion	80 011	84 923	1,1	1 302	3,50
	2-opt	90 544	99 937	3,8	2 466	3,58

Tabela 3. Variantes do GD com Insertion (orçamento e runs do T3). Passos de melhoria e ótimos locais por run; aval./passo = B/passos; $h^\star$=1875: runs que atingiram o ótimo conhecido.

Inst.	Variante	$\tilde{h}^\star$	passos	aval./passo	ót. loc.	$h^\star$=1875
Pequena	Best	1 875	364	137	48,2	30/30
	First	1 875	2 588	19	147,1	30/30
	Aleatório	1 875	2 640	19	154,5	30/30
Intermediária	Best	19 998	61	2 459	1,0	–
	First	16 990	1 917	78	12,2	–
	Aleatório	17 194	1 884	80	11,7	–
Grande	Best	822 160	13	23 077	0,0	–
	First	80 011	1 302	230	1,1	–
	Aleatório	84 270	1 428	210	1,4	–

caixeiro-viajante, mas aqui reordena prazos que já estavam bem sequenciados. A vizinhança maior da Insertion faz com que cada descida atinja menos ótimos locais dentro de B, mas melhores. Por isso ela foi adotada como vizinhança padrão do GD e do SA.

2.2 Greedy Descent com Random Restart

O GD parte de uma permutação aleatória e só aceita movimentos que reduzem h. Implementamos três variantes. O *Best Improvement* avalia toda a vizinhança e move para o melhor vizinho. O *First Improvement* percorre a vizinhança numa ordem embaralhada, sorteada a cada reinício e percorrida de forma circular, e move para o primeiro vizinho que melhora h. O *sucessor aleatório* sorteia vizinhos com reposição e aceita o primeiro que melhora. Nas duas primeiras variantes, s é um ótimo local quando $|V|$ vizinhos seguidos não melhoram h. Na terceira, o mesmo critério só garante otimalidade local com alta probabilidade. Ao atingir um ótimo local, o algoritmo o registra e reinicia de uma nova permutação aleatória, até esgotar B. A melhor solução global é mantida à parte. A solução corrente nunca é reavaliada: seu h é conhecido, e cada vizinho custa uma avaliação.

Passos dentro do orçamento. A Tabela 3 mostra o efeito do tamanho da vizinhança. O Best Improvement paga $|V|$ avaliações por passo: em $n = 150$, cada varredura custa 22 201 avaliações, e $B = 300\,000$ permite apenas 13 passos de melhoria a partir de uma solução aleatória. A primeira descida nem termina, e a mediana final (822 160) é dez vezes pior que a do First Improvement (80 011). O First e o aleatório dão $\approx 1\,300$ passos com ≈ 200 avaliações por passo, porque aceitam a primeira melhora encontrada. O First Improvement foi o melhor nas instâncias média e grande e é a configuração do GD no T3.

Ótimos locais na instância pequena. Em $n = 12$, as três variantes atingem o ótimo conhecido $h = 1\,875$ em todas as 30 runs. Ao longo dos reinícios, o First Improvement visitou 4 413 ótimos locais (147 por run), mas apenas **19 permutações distintas**, a mesma contagem do Best Improvement. A paisagem da Insertion em $n = 12$ tem, portanto, poucos ótimos locais, e 14% das descidas terminam no ótimo global. Com ≈ 150 reinícios por run, a chance de nunca cair na bacia do ótimo é desprezível ($0{,}86^{150} \approx 10^{-10}$). O sucessor aleatório declara 391 "ótimos" distintos. Pelo critério probabilístico, parte deles não são ótimos locais de fato, e só 9% das suas descidas terminam em 1 875.

2.3 Simulated Annealing e Calibração Térmica

O *Simulated Annealing* [1, 2] é uma busca local estocástica que escapa de ótimos locais ao aceitar, com probabilidade controlada, movimentos que pioram a solução. A cada passo sorteia-se uniformemente um par ordenado (origem, destino) de posições distintas, o que define um vizinho π' da solução corrente π na vizinhança Insertion, e calcula-se $\Delta h = h(\pi') - h(\pi)$. A transição segue o critério de Metropolis, derivado da distribuição de Gibbs $P(\pi) \propto e^{-h(\pi)/T}$:

$$P(\pi \to \pi') = \begin{cases} 1, & \Delta h \le 0, \\ e^{-\Delta h/T}, & \Delta h > 0. \end{cases} \tag{2}$$

Como a proposta é simétrica (o movimento inverso tem a mesma probabilidade de ser sorteado), para T fixo a cadeia de Markov resultante tem a distribuição de Gibbs como distribuição estacionária. Quando $T \to \infty$, todo vizinho é aceito (*random walk*). Quando $T \to 0$, a massa de probabilidade se concentra nos mínimos de h, e a regra (2) se reduz a uma descida gulosa estocástica. O resfriamento leva a busca de um regime ao outro. A melhor solução global $\pi^\star$ é mantida à parte, porque o estado corrente pode terminar pior do que algum estado já visitado.

Calibração empírica de T_0. A escala de Δh depende fortemente de n. Os atrasos se acumulam ao longo da rota, de modo que reinserir um ativo desloca os instantes C_k de todo o sufixo. Uma temperatura fixa que funcione em $n = 12$ seria fria demais em $n = 150$, e uma adequada a $n = 150$ seria um *random walk* em $n = 12$. Por isso T_0 é calibrado em cada execução: a partir do estado inicial aleatório, sorteiam-se 200 vizinhos, calcula-se a média $\overline{\Delta h}^{+}$ apenas dos aumentos de custo ($\Delta h > 0$) e impõe-se que, no início, uma piora típica seja aceita com probabilidade $p_0 = 0{,}80$:

$$e^{-\overline{\Delta h}^{+}/T_0} = p_0 \implies T_0 = \frac{-\overline{\Delta h}^{+}}{\ln p_0} \approx 4{,}48\,\overline{\Delta h}^{+}. \tag{3}$$

A Tabela 4 mostra que $\overline{\Delta h}^{+}$ cresce cerca de 15× entre $n = 12$ e $n = 150$, e T_0 acompanha esse crescimento sem nenhum ajuste manual. As 200 avaliações de calibração são debitadas do orçamento B.

Esquema geométrico. A temperatura decai a cada avaliação, $T_{k+1} = \alpha\, T_k$, com

$$T_f = \rho\, T_0, \qquad \alpha = \left(\frac{T_f}{T_0}\right)^{1/L} = \rho^{1/L}, \tag{4}$$

em que $L = B - 201$ é o número de avaliações que restam após a avaliação do estado inicial e a calibração. Assim, T atinge

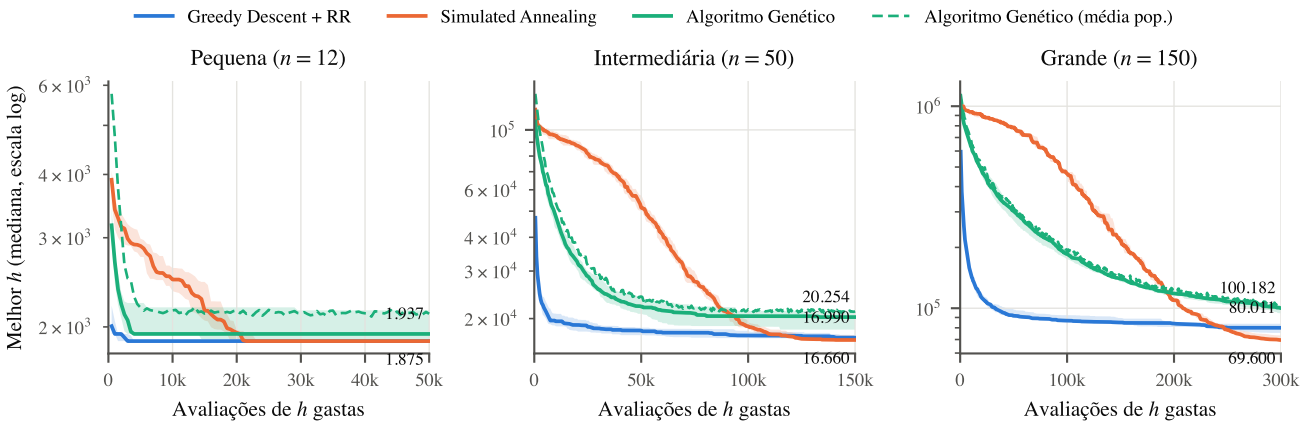


Figura 1. Curvas de convergência: mediana do melhor h (escala log) em função das avaliações gastas, com faixa interquartil, para as três instâncias. Tracejado: mediana do h médio da população do GA.

Tabela 4. Parâmetros do SA calibrados por instância (mediana entre as runs) e qualidade final. $\overline{\Delta h}^{+}$: piora média nos 200 vizinhos iniciais; $h^{\star}_{\min}$ e $\tilde{h}^{\star}$: mínimo e mediana do melhor h final.

n	B	runs	$\overline{\Delta h}^{+}$	T_0	T_f	α	$h^{\star}_{\min}$	$\tilde{h}^{\star}$
12	50 000	30	603	2 703	2,7	0,9998613	1 875	1 875
50	150 000	30	3 492	15 651	15,7	0,9999539	16 179	16 660
150	300 000	10	8 872	39 759	39,8	0,9999770	66 238	69 600

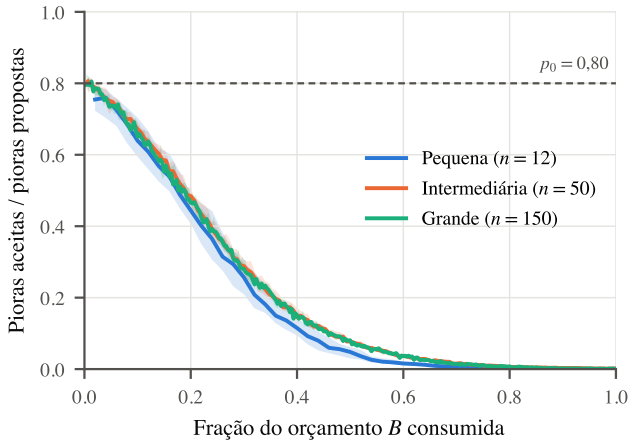


Figura 2. Fração das propostas de piora aceitas pelo SA em janelas de 1 000 avaliações, em função da fração consumida do orçamento. Linha: mediana entre as runs; faixa: intervalo interquartil.

T_f exatamente quando o orçamento se esgota. Adotamos $\rho = 10^{-3}$. Em testes-piloto com 3 a 4 sementes (fora do conjunto oficial), fixar T_f pela mesma régua de T_0 (isto é, $T_f = -\overline{\Delta h}^{+}/\ln p_f$ com $p_f \in [10^{-2}, 10^{-4}]$) deixava o fim da busca quente demais em $n = 150$ (medianas acima de $1{,}5 \times 10^5$). Isso ocorre porque $\overline{\Delta h}^{+}$ é medido num estado aleatório, cujas piores são muito maiores que as observadas perto de bons ótimos locais. Com $\rho = 10^{-3}$, T_f fica entre 2,7 e 39,8 unidades de custo (Tabela 4), da ordem das menores piores possíveis. Valores menores ($\rho = 10^{-4}$) não trouxeram ganho. A vizinhança é a Insertion, escolhida na T2 (Seção 2.1).

Diagnóstico térmico. Em cada janela de 1 000 avaliações registramos a razão entre as piores aceitas e as piores propostas (Figura 2). As três curvas partem de $\approx 0{,}80$, o que confirma a calibração (3), e caem de forma suave e praticamente monotônica: a taxa mediana passa de 0,5 em $\approx 18\%$ do orçamento, de 0,2 em $\approx 35\%$ e de 0,05 entre 50% e 57%, e fica abaixo de 1% a partir de $\approx$ 66–75%. O primeiro terço da busca é, portanto, exploratório: a solução corrente perambula por regiões de custo alto, o que aparece como a descida lenta no início das curvas de convergência das instâncias média e grande (Figura 1). O terço final é, na prática, uma descida gulosa estocástica que refina a bacia escolhida. O fato mais relevante é que as três curvas quase se sobrepõem quando o eixo horizontal é normalizado por B. A calibração por $\overline{\Delta h}^{+}$ remove a dependência de escala, e a forma do resfriamento passa a depender apenas da fração do orçamento consumida, não de n. Não se observa nenhum dos dois modos de falha clássicos: nem um platô longo perto de 1 (temperatura alta demais, que desperdiça orçamento em passeio aleatório) nem uma queda abrupta para 0 no início (têmpera rápida demais, que congela a busca num ótimo local ruim).

2.4 Algoritmo Genético e Operadores de Permutação

No Algoritmo Genético (`src/genetic_algorithm.py`), cada cromossomo é uma permutação completa $\pi = (\pi_1, \ldots, \pi_n)$ precedida pela base $\pi_0 = 0$. A população inicial de tamanho $N = 100$ é gerada por amostragem uniforme. Como o problema é de minimização de $h(\pi)$, convertemos o custo na aptidão estritamente decrescente $f(\pi) = 1/(1 + h(\pi))$. A seleção de pais usa **torneio de tamanho** $k = 3$ (garantindo dois pais de índices distintos): sorteiam-se 3 indivíduos e escolhe-se o de maior $f(\pi)$, o que equivale diretamente a escolher o de menor custo $h(\pi)$ (o módulo também suporta roleta proporcional a $f(\pi)$). A cada geração aplicamos **elitismo** copiando os $E = 2$ melhores indivíduos sem reavaliar h, e geramos os $N - E = 98$ filhos restantes.

Cruzamento OX, mutação e diversidade. Com probabilidade $P_c = 0{,}85$, dois pais geram um filho pelo *Order Crossover* (OX), que preserva a validade da permutação sem duplicar ativos; caso contrário, o filho copia o primeiro pai. Por exemplo, para $n = 6$, $\text{Pai}_1 = (1, 4, 3, 5, 2, 6)$, $\text{Pai}_2 = (6, 3, 1, 2, 4, 5)$ e corte nas posições 2 a 4: (i) copia-se o bloco $[4, 3, 5]$ de Pai_1, formando $(_, 4, 3, 5, _, _)$; (ii) percorre-se Pai_2 circularmente a partir da posição 5, filtrando os ativos ausentes na ordem em que ocorrem: $[4, 5, 6, 3, 1, 2] \setminus \{4, 3, 5\} = [6, 1, 2]$; (iii) preenchem-se as posições 5, 6, 1, resultando no filho válido $(2, 4, 3, 5, 6, 1)$. Em seguida, com probabilidade $P_m = 0{,}20$, aplica-se uma mutação por *Insertion* (vizinhança escolhida na T2). O orçamento B corresponde a 509, 1 529 e 3 060 gerações completas

($n = 12, 50, 150$). Monitorando a fração de permutações distintas a cada geração, diagnostica-se convergência prematura: sob torneio $k = 3$, a diversidade cai de 100% para $\approx 20\%$ já em 10% do orçamento (≈ 50 gerações) em $n = 12$ e em 50%–60% do orçamento em $n = 50$ e 150, estabilizando em 18%–24% (exatamente a fração de filhos mutados por geração, $P_m = 0{,}20$).

3 Metodologia Experimental e Protocolo T3

Orçamento por avaliações, não por tempo. O critério de parada de todos os algoritmos é um orçamento B de *avaliações da função objetivo*. Cada chamada a $h(\cdot)$ consome exatamente uma unidade, seja ela uma avaliação completa da Eq. (1) ou incremental. São contadas a avaliação da solução inicial, as 200 avaliações de calibração do SA, cada vizinho examinado pelo GD (inclusive os rejeitados) e cada indivíduo avaliado pelo GA. A contagem é centralizada em um único objeto `Avaliador`, encapsulado pela interface comum `ProblemaManutencao` (`src/problema.py`), pela qual passam todas as chamadas a h e que levanta uma exceção ao se tentar ultrapassar B. Assim, nenhum algoritmo consegue gastar mais do que o orçamento. O orçamento por avaliações é a métrica de esforço adotada na comparação porque independe da máquina, da carga do sistema e da qualidade da implementação. Dois métodos com o mesmo B examinaram o mesmo número de soluções candidatas. O tempo de parede (`time.perf_counter`) é registrado à parte, como medida secundária. Ele capta diferenças que o orçamento esconde: nem toda avaliação custa o mesmo (a incremental percorre só as posições $i, \ldots, n$), e as operações genéticas do GA têm custo próprio, que não passa por h. No lote T3, o tempo de parede por avaliação foi de 2,6, 5,7 e 13,1 μs no GD, de 3,0, 7,6 e 18,4 μs no SA e de 8,0, 14,3 e 31,4 μs no GA ($n = 12$, 50, 150). O SA é mais caro por avaliação que o GD porque calcula $e^{-\Delta h/T}$, sorteia a aceitação e reconstrói C e H a cada movimento aceito, enquanto o GD só reconstrói ao melhorar e usa poda. Já o GA é o mais caro por avaliação (2,4× a 3,1× o GD) porque todo filho requer avaliação completa desde a posição 1, somada ao custo de seleção por torneio, cruzamento OX e ordenação da elite. Esses tempos são indicativos: num notebook sob carga contínua, o *throttling* térmico altera o tempo de parede em até $\approx 30\%$ (uma run isolada do SA em $n = 50$ leva 0,87 s, contra 1,14 s de média no lote). Por isso o orçamento em avaliações é a medida de esforço da comparação.

Configurações. As nove combinações (3 algoritmos × 3 instâncias) usam $B = 50\,000$, 150 000 e 300 000 avaliações e 30, 30 e 10 runs independentes nas instâncias pequena ($n = 12$), intermediária ($n = 50$) e grande ($n = 150$), respectivamente. Cada execução parte de uma permutação aleatória e grava, a cada 500 avaliações, o melhor h encontrado até ali, o que dá $B/500$ pontos por curva de convergência. O SA grava ainda T e a taxa de aceitação de pioras a cada 1 000 avaliações.

Controle por sementes. A run r de qualquer algoritmo usa a semente $s_r = 20\,260\,000 + r$, e toda a aleatoriedade de uma execução (permutação inicial, sorteio de vizinhos, aceitação, operadores genéticos) vem de um único gerador `random.Random`(s_r) local. Nenhum algoritmo usa o gerador global do módulo `random`. Com isso, (i) qualquer execução é reproduzida bit a bit a partir de sua semente, e (ii) os três algoritmos usam as mesmas 30 (ou 10) sementes em cada instância. Usar a mesma semente, porém, não torna as runs pareadas: cada algoritmo consome o gerador de forma diferente, de modo que GD, SA e GA na run r não compartilham solução inicial nem trajetória. As amostras são, portanto, tratadas como independentes, e as comparações usam o teste de Mann–Whitney (Seção 4).

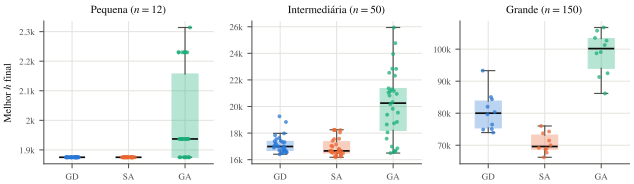


Figura 3. Distribuição do melhor h final por algoritmo e instância.

Execução e registro. As runs são executadas sequencialmente pelo orquestrador `scripts/experimentos_t3.py`, em um único processo, para que o tempo de parede não sofra contenção de CPU. Ele grava cada run em `resultados_t3.csv` imediatamente após seu término: algoritmo, instância, *run*, semente, $h^\star$ final, tempo de parede e histórico de convergência. Uma interrupção não perde as runs concluídas, e uma nova chamada retoma das pendentes. Os parâmetros calibrados do SA e o diagnóstico de aceitação vão para `parametros_sa.csv` e `diagnostico_sa.csv`. Todas as figuras e tabelas deste relatório são regeneradas a partir desses arquivos por `scripts/plot_graficos.py`. Os experimentos foram executados em Python 3.11.9, em um único núcleo de um Intel Core i7-1255U.

4 Resultados Comparativos e Análise Estatística

A Tabela 5 e os boxplots da Figura 3 sintetizam as 9 combinações do protocolo T3. Na instância pequena ($n = 12$), tanto o GD quanto o SA encontram o ótimo exato $h^\star = 1\,875$ em 30/30 runs (gap = 0%, $p = 1{,}00$ no teste bicaudal de Mann–Whitney). O GA atinge 1 875 em apenas 9/30 runs (30%) e fica retido nos demais ótimos locais da Insertion ($h = 1\,937$ em 13/30 runs e $h = 2\,230$ em 7/30 runs, estagnando em mediana após 8% de B), fechando com mediana 1 937, média 1 999 (gap = 6,63%) e diferença estatisticamente significativa frente a GD e SA ($p = 3{,}7 \times 10^{-8}$). Na instância intermediária ($n = 50$), o SA obtém mediana 16 660 contra 16 990 do GD ($p = 0{,}07$, empate estatístico ao nível de 5%) e **alcança mínimo $h^\star_{\min} = 16\,179$, superando o melhor valor de referência do enunciado** (16 332); ambos superam amplamente o GA (mediana 20 254, $p = 2{,}4 \times 10^{-7}$ para SA×GA e $p = 8{,}6 \times 10^{-6}$ para GD×GA). Na instância grande ($n = 150$), o SA assume liderança isolada e estatisticamente significativa sobre o GD (mediana 69 600 contra 80 011, $p = 5{,}8 \times 10^{-4}$, com mínimo 66 238 próximo à referência 60 417), enquanto o GA fica em terceiro lugar (100 182, $p = 1{,}8 \times 10^{-4}$ contra SA e $p = 4{,}4 \times 10^{-4}$ contra GD).

5 Discussão: Escala, Esforço Computacional e Conclusões

Efeito da escala no SA. Na instância pequena, o SA atinge o ótimo conhecido $h = 1\,875$ em todas as 30 runs, em

Tabela 5. Resumo do melhor h final por combinação algoritmo–instância, gap relativo $(\bar{h} - 1875)/1875$ na instância pequena, orçamento B e tempo médio de parede $\bar{t}$ por run. Valores de referência do enunciado: 1 875 ($n = 12$), 16 332 ($n = 50$) e 60 417 ($n = 150$).

Inst.	Alg.	runs	média	d.p.	mín.	mediana	gap	B	$\bar{t}$ (s)
Pequena	GD	30	1 875	0	1 875	1 875	0,00%	50 000	0,13
Pequena	SA	30	1 875	0	1 875	1 875	0,00%	50 000	0,15
Pequena	GA	30	1 999	151	1 875	1 937	6,63%	50 000	0,40
Intermediária	GD	30	17 152	656	16 408	16 990	–	150 000	0,85
Intermediária	SA	30	16 944	673	16 179	16 660	–	150 000	1,14
Intermediária	GA	30	20 145	2 561	16 498	20 254	–	150 000	2,14
Grande	GD	10	80 515	5 988	73 957	80 011	–	300 000	3,94
Grande	SA	10	70 628	3 153	66 238	69 600	–	300 000	5,52
Grande	GA	10	98 780	6 757	86 176	100 182	–	300 000	9,43

Tabela 6. Experimento adicional do GA (10 runs): cruzamento OX ligado ($P_c = 0{,}85$) × desligado ($P_c = 0$), p-valor de Mann–Whitney, fração de cruzamentos em que o filho supera ambos os pais ou ao menos um pai, e diversidade final.

Inst.	$\tilde{h}^\star$ (OX)	$\tilde{h}^\star$ (P_c=0)	p-valor	filho < pais	filho < 1 pai	div. final
Pequena	1 937	2 084	0,52	0,29%	2,3%	18,0%
Intermediária	20 390	19 930	0,97	0,60%	6,3%	22,0%
Grande	100 182	99 152	0,68	0,98%	11,0%	23,9%

mediana após ≈ 21 mil avaliações (43% de B). O restante do orçamento é gasto sem melhoria. Nas instâncias média e grande, ao contrário, a curva mediana só entra a 5% do valor final após 74% e 88% do orçamento, respectivamente, e a dispersão entre runs cresce: o desvio-padrão relativo vai de 0 para 4% ($n = 50$) e 4,5% ($n = 150$). Isso é coerente com o crescimento do espaço de busca ($n!$) e da vizinhança Insertion ($(n-1)^2 = 22\,201$ vizinhos em $n = 150$). Com $B = 300\,000$, o SA examina o equivalente a apenas ≈ 13 varreduras completas da vizinhança, e a curva mediana ainda desce quando o orçamento acaba. Isso sugere que, em $n = 150$, o fator limitante é o orçamento, e não o esquema de temperatura.

GD × SA × GA: ótimos locais, escala e exploração/intensificação. O ranking muda com n (Tabela 5 e Figura 1). Em $n = 12$, a paisagem da Insertion possui apenas 19 ótimos locais distintos (Seção 2.2): o GD (por executar ≈ 147 reinícios independentes por run) e o SA (por escapar de vales rasos via temperatura) alcançam o ótimo global $h^\star = 1\,875$ em 30/30 execuções (6% e 43% de B, respectivamente), enquanto o GA só o atinge em 9/30 execuções porque sua população colapsa precocemente sobre um único ótimo local (1 937 ou 2 230) sem mecanismo de reinício. Em $n = 50$, o SA termina ligeiramente à frente do GD (mediana 16 660 contra 16 990). Em $n = 150$, a vantagem do SA é clara: 69 600 contra 80 011, uma diferença de 13%, com metade da dispersão. As curvas mostram o mecanismo. No GD, a intensificação é a descida gulosa e a exploração vem só dos reinícios: em $n = 150$, cada descida consome ≈ 230 avaliações por passo e cabe pouco mais de um reinício em B, extinguindo a exploração por reinício. No SA, a exploração é a aceitação de pioras controlada por T, que escapa de ótimos locais medianos sem descartar o progresso acumulado; em $n = 12$, ao contrário, reiniciar é a forma mais barata de explorar os 19 ótimos locais.

Quando o GA compensa e experimento sobre recombinação. No GA, a exploração cabe à diversidade populacional e à mutação ($P_m = 0{,}20$), e a intensificação cabe à seleção por torneio e ao cruzamento OX. Nas três instâncias, a curva tracejada da média da população (Figura 1) aproxima-se rapidamente da curva do melhor indivíduo, refletindo a queda da fração de indivíduos únicos para ≈ 20% (Tabela 6). Nossa hipótese para o desempenho inferior do GA é que o operador OX, embora preserve a ordem relativa, **quebra os instantes acumulados** C_k: como o atraso $\max\{0, C_k - d_{\pi_k}\}$ depende da soma de todos os tempos anteriores, um bloco de ativos bem sequenciado em Pai_1, ao ser deslocado pelo prefixo herdado de Pai_2, passa a violar prazos. Para testar essa hipótese diretamente (`scripts/tarefa_ga_experimento.py`), comparamos o GA padrão ($P_c = 0{,}85$) com o mesmo GA sem cruzamento ($P_c = 0$, apenas seleção e mutação). A Tabela 6 confirma a hipótese: desligar o OX não altera significativamente o resultado em nenhuma instância ($p \geq 0{,}52$), e menos de 1% dos filhos gerados por OX superam ambos os pais. Conclui-se que, sob orçamento fixo e forte acoplamento temporal no prefixo, buscas locais baseadas na vizinhança Insertion com avaliação incremental (**GD** em pequena escala e **SA** em média e grande escala) são mais eficazes e até 2,8× mais rápidas em tempo de parede do que o GA puramente geracional.

Referências

[1] S. Kirkpatrick, C. D. Gelatt e M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[2] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller e E. Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[3] S. Russell e P. Norvig. *Artificial Intelligence: A Modern Approach*. 4. ed., Pearson, 2021.